

非線形モデル予測制御の自動コード生成ツール

大塚 敏 之*

* 京都大学 大学院情報学研究科 京都府京都市左京区吉田本町
* Graduate School of Informatics, Kyoto University, Yoshida-honmachi,
Sakyo-ku, Kyoto, Japan
* E-mail: ohtsuka@i.kyoto-u.ac.jp

キーワード：モデル予測制御 (model predictive control), 最適制御 (optimal control), 非線形システム (nonlinear system), 自動コード生成 (automatic code generation), 並列計算 (parallel computation).

JL 0009/23/6209-0536 ©2023 SICE

1. はじめに

近年、モデル予測制御 (Model Predictive Control: MPC) がさまざまな分野で注目されている。MPC は各時刻で有限時間未来まで動的システムの応答を最適化し制御入力決定するフィードバック制御手法である。制約条件や非線形性をもつ動的システムであっても最適化問題さえ制御周期内に解ければ適用できる汎用性が MPC の魅力となっている。たとえば、Boston Dynamics のヒューマノイドロボット Atlas¹⁾ や Toyota Research Institute のドリフト走行する自動運転車両²⁾ に MPC が使われている。また、機械学習のトップカンファレンス NeurIPS で MPC のチュートリアル³⁾ が行われたり、ロボティクス分野で論文特集号⁴⁾ が組まれたりしている。非線形制御に関する本誌の特集号⁵⁾ では、非線形モデル予測制御 (Nonlinear MPC: NMPC) が実用的な非線形制御手法の 1 つに挙げられている。MPC の問題設定や発展の歴史については、過去の解説記事⁶⁾ を参考にされたい。

MPC とくに NMPC が注目され応用が広がっている背景には計算機と数値解法の進歩がある。しかし、高度な数値解法を適用対象に合せて実装するのは容易でない。そのため、プログラミングを自動化する自動コード生成などのソフトウェアツールが NMPC では重要になっており、活発に開発されている^{6)~8)}。本稿では、多くのソフトウェアツールを概観することよりも読者が実際に試せることに主眼を置き、現在の計算機環境で標準的なマルチコア CPU を活用できる NMPC コード生成ツール ParNMPC^{9), 10)} に焦点を当てて解説する。

2. 並列計算コード生成ツール ParNMPC

2.1 数値解法の概要

ParNMPC は Deng Haoyang 氏によって開発され GitHub で無償公開されている¹¹⁾。2 条項 BSD ライセンスのもとで商用を含めた利用が可能である。ParNMPC が扱う問題設定は、各時刻で以下のような最適制御問題を解く NMPC である。

$$\min_{x(\cdot), u(\cdot)} \int_0^T L(u(\tau), x(\tau), p(\tau)) d\tau \quad (1)$$

$$\text{s.t. } x(0) = \bar{x}_0, \quad (2)$$

$$M(u(\tau), x(\tau), p(\tau)) \dot{x}(\tau) = f(u(\tau), x(\tau), p(\tau)), \quad \tau \in [0, T], \quad (3)$$

$$C(u(\tau), x(\tau), p(\tau)) = 0, \quad \tau \in [0, T], \quad (4)$$

$$G(u(\tau), x(\tau), p(\tau)) \geq 0, \quad \tau \in [0, T]. \quad (5)$$

ここで、(1) 式が最小化すべき評価関数であり、 $u(\tau)$ は制御入力ベクトル、 $x(\tau)$ は制御対象とする動的システムの状態ベクトル、 $p(\tau)$ は時変パラメータのベクトルである。(3) 式が制御対象の状態方程式であり、非線形システムでもよい。さらに、左辺の $\dot{x}(\tau)$ に正則な行列 $M(u, x, p)$ がかかっている場合も扱える。また、(4)、(5) 式はそれぞれ等式制約条件と不等式制約条件である。不等式制約条件の $G(u, x, p)$ は u, x の 1 次式だと仮定するが、入力の一部をスラック変数とすれば一般的な不等式制約条件も扱える。最適制御問題の初期状態 $x(0)$ は (2) 式のように $\bar{x}_0$ で与えられる。ただし、MPC の場合、各時刻 t において制御対象の状態 $x(t)$ を計測して $\bar{x}_0$ として与え、 T だけ未来までの最適制御問題を解いて最適制御入力 $u(\tau), \tau \in [0, T]$ を求め、その初期値のみを実際の制御入力 $u(t)$ としてシステムに加えることになる。したがって、評価区間上の時刻 $\tau \in [0, T]$ は実際のシステムにおける時刻 $t + \tau$ に対応する。

上記のような最適制御問題の停留条件は非線形 2 点境界値問題となり、勾配法やニュートン法などの反復解法によって停留条件を満たす制御入力の数値解を求めるのが一般的である。NMPC を実装するには制御周期内で数値解を求める必要があり、NMPC に特化した効率的な数値解法が活発に研究されている⁶⁾。現在の計算機ではマルチコア CPU が一般的なので、並列計算を活用した計算の高速化も当然考えられる。そのためには、最適制御問題を分割して並列処理したい。とくに、最適制御問題の数値解法では評価区間 $[0, T]$ を多数の時間ステップに離散化し各時間ステップでの変数を計算するので、時間ステップごとの変数を計算する処理を完全に並列化できれば大幅な高速化が期待できる。理想的には評価区間をいくら長くしてもコア数さえ増やせば計算時間が増えないことが望ましい。しかし、動的システムの場合、過去の入力や状態が未来の状態に影響するという因果関係があるため、時間方向の並列化は一見難しい。

それに対し ParNMPC では、隣り合う時間ステップ間

の変数の感度計算に必要最小限の近似を導入することで時間方向の並列化を実現している。具体的には、ニュートン法による解の更新の際に必要な感度を厳密に計算するのではなく、1 回前の更新で計算した感度によって代用することで、各時間ステップの変数を更新する際に必要となる行列を並列計算可能にしている。たとえば線形システムに対する 2 次評価関数であれば感度は変化しないので、非線形システムの一般的な問題設定であっても感度の変化は小さいと期待でき、実際に ParNMPC の反復解法は超 1 次収束することが示されている¹²⁾。そのほか、並列化に適した時間離散化やヘッセ行列の近似、不等式制約条件を扱う内点法の工夫なども用いている⁹⁾。

2.2 使用環境と処理の流れ

ParNMPC の使用には MATLAB (R2016a 以降) のほか、MATLAB Coder, Parallel Computing Toolbox や Symbolic Math Toolbox などのツールボックス、そして、並列計算用 API OpenMP に対応した C コンパイラが必要である。MATLAB のコマンドラインで `mex-setup` を実行して MATLAB のコード生成で使用する既定の C コンパイラを確認し、変更する場合は表示される指示に従って設定しておく。詳細についてはホームページ¹⁰⁾ を参照されたい。本稿で紹介するのは、MATLAB R2023a と C コンパイラ Microsoft Visual Studio 2022 を OS Windows 11 Pro 上で使用した場合である。使用した計算機は CPU Intel Core i7-1165G7 2.80 GHz 4 コア (8 スレッド)、RAM 32 GB である。



図 1 ParNMPC のフォルダ構成

ParNMPC は共通のファイル、docs はドキュメント、ほかは例題を格納している。

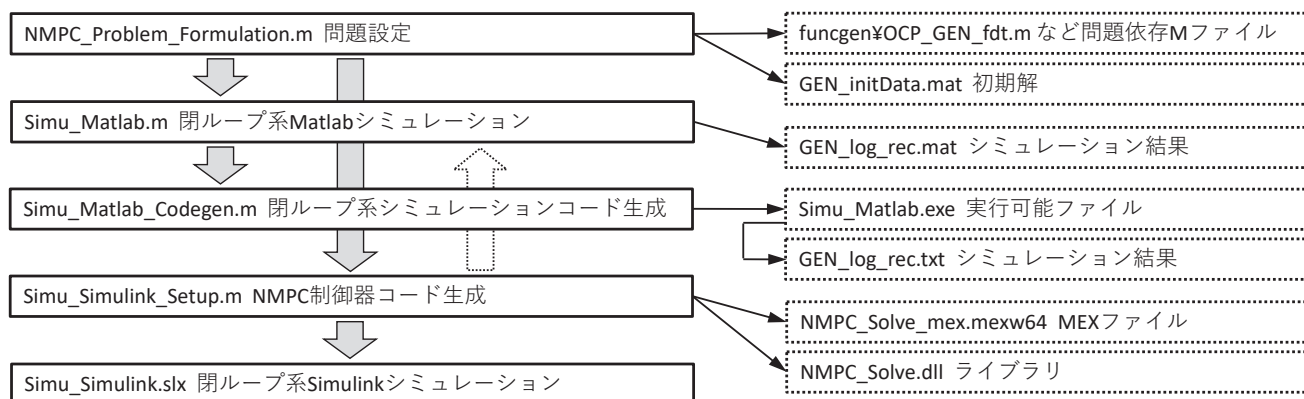


図 2 問題ごとに使用するファイル

実線四角のファイルを実行すると点線四角のファイルが生成される。細矢印はファイルの生成を表わし、太矢印は生成されたファイルの使用など依存関係を表わす。点線の太矢印は生成された MEX ファイルを MATLAB シミュレーションで利用できることを表わす。

ParNMPC のインストールは、GitHub のサイト¹¹⁾ からファイル一式をダウンロードし適当な作業用フォルダにコピーするだけでよい。フォルダ構成は図 1 のようになっている。ParNMPC フォルダには問題に依存しない共通の MATLAB M ファイルが、docs フォルダにはドキュメントがそれぞれ格納されている。その他のフォルダは例題であり、それぞれの問題に対して使用する M ファイルが格納されている。

図 2 に、例題のフォルダに格納されている主な M ファイル (実線四角) と処理の流れ (太矢印)、生成されるファイル (細矢印と破線四角) を示す。まず、NMPC_Problem_Formulation.m に状態方程式 (3) や評価関数 (1) などの問題設定を書き込み実行すると、数式処理が行われ問題依存の M ファイルなどが生成される。つづいて Simu_Matlab.m を実行すると、生成された M ファイルを使用した NMPC の閉ループ系シミュレーションが実行される。さらに、Simu_Matlab_Codegen.m を実行すると実行可能ファイル Simu_Matlab.exe が生成され、同じシミュレーションがより高速に実行できる。

Simu_Simulink_Setup.m は、Simulink シミュレーションで使用する制御器用の NMPC ソルバをライブラリとして生成し、ブロック線図 Simu_Simulink.slx のパラメータを設定する。Simu_Simulink.slx を実行すると、Simulink 上で NMPC の閉ループ系シミュレーションが実行される。また、Simu_Simulink_Setup.m の設定を変更すれば NMPC ソルバの MEX 関数が生成でき、その MEX 関数を Simu_Matlab.m から呼び出すことも可能である。

2.3 問題設定とコード生成

以下では、Quadrotor フォルダに格納されているクワッドローターの例題に沿って使用方法を説明する。MATLAB のカレントディレクトリもそのフォルダに移動しているものとする。M ファイル NMPC_Problem_Formulation.m の冒頭部分を図 3 に示す。ただし改

```

1 clear all
2 addpath(' ../ParNMPC/')
3
4 % Create an OptimalControlProblem object
5 OCP = OptimalControlProblem(5,9,3,... % dimensions of inputs, states, and parameters
6                                48); % N: number of discretization grids
7
8 % Give names to x, u, p
9 [X,dX,Y,dY,Z,dZ,Gamma,Beta,Alpha] = ...
10 OCP.setStateName({'X','dX','Y','dY','Z','dZ','Gamma','Beta','Alpha'});
11 [a,omegaX,omegaY,omegaZ,slack] = ...
12 OCP.setInputName({'a','omegaX','omegaY','omegaZ','slack'});
13 [XSP,YSP,ZSP] = OCP.setParameterName({'XSP','YSP','ZSP'},[1 2 3]);
14
15 OCP.setT(2); % Set the prediction horizon T
16
17 % Set the dynamic function f
18 g = 9.81;
19 f = [ dX; a*(cos(Gamma)*sin(Beta)*cos(Alpha) + sin(Gamma)*sin(Alpha));...
20       dY; a*(cos(Gamma)*sin(Beta)*sin(Alpha) - sin(Gamma)*cos(Alpha));...
21       dZ; a*cos(Gamma)*cos(Beta) - g;...
22       (omegaX*cos(Gamma) + omegaY*sin(Gamma))/cos(Beta);...
23       -omegaX*sin(Gamma) + omegaY*cos(Gamma);...
24       omegaX*cos(Gamma)*tan(Beta) + omegaY*sin(Gamma)*tan(Beta) + omegaZ];
25 OCP.setf(f);
26 OCP.setDiscretizationMethod('Euler');

```

図3 NMPC_Problem_Formulation.m の例

制御対象の状態方程式や評価関数などを定義する M ファイルであり、実行すると問題依存の M ファイルが生成される。

行位置を調整している。5～6 行において `OptimalControlProblem` クラスのオブジェクト `OCP` を定義している。引数は、入力 u 、状態 x 、パラメータ p それぞれの次元および評価区間の離散化ステップ数 N である。8～12 行では、状態その他の変数に対して名前を設定している。これらの変数は数式処理用のシンボリック変数である。変数名の設定は省略可能であり、その場合は `OCP` オブジェクトのプロパティ `OCP.x(1)`、`OCP.x(2)` などがデフォルトのシンボリック変数として使用できる。14 行目の `OCP.setT(2)` は評価区間長さ T を引数として与えて設定するメソッドである。18～23 行では、上で定義したシンボリック変数を用いた状態方程式 (3) 右辺の数式を配列 f に代入しており、24 行目のメソッド `OCP.setf(f)` で `OCP` オブジェクトのプロパティとして設定している。この例題では (3) 式左辺の行列 M が単位行列であり省略されている。状態方程式に含まれるパラメータ g は数値を代入する通常の変数として定義されている。必要なら行列も使用可能である。25 行目の `OCP.setDiscretizationMethod('Euler')` は状態方程式の離散化方法を後退オイラー法に指定している。

図 3 では省略しているが、上記の処理に続いて `OCP.setL(L)` と `OCP.setG(G)` によって同様に評価関数 (1) と不等式制約条件 (5) をそれぞれ設定する。その後、`OCP.codeGen()` によって問題依存の M ファイルをいくつか生成し `funcgen` フォルダに保存する。続いて、`NMPCSolver` オブジェクト `nmpcSolver` を定義し、ヘッ

セ行列近似の設定を行った後に `nmpcSolver.codeGen()` でソルバ用の M ファイルを生成し `funcgen` フォルダに保存する。

`NMPC_Problem_Formulation.m` の末尾では、閉ループ系シミュレーション開始時の最適解を計算、保存するとともに、シミュレーション用の制御対象システムを定義する。シミュレーションの初期状態 x_0 と初期推定解を設定して制御開始時の最適解を計算し、初期状態などの設定を MAT ファイル `GEN_initData.mat` に保存するとともに、グローバル変数 `ParNMPCGlobalVariable` に最適解を保存する。その後、シミュレーションにおける制御対象システムを `DynamicalSystem` クラスのオブジェクト `plant` として定義する。定義の方法は前述の最適化計算用システム f と同様である。これによって、最適化計算用モデルと実際の制御対象システムとが異なる場合でも閉ループ系シミュレーションが行える。

2.4 MATLAB によるシミュレーション

`NMPC_Problem_Formulation.m` を実行し問題依存の M ファイルが生成されたら、`Simu_Matlab.m` によって NMPC の閉ループ系シミュレーションが実行できる。シミュレーションの時間を変数 `simuLength` に、時間刻みを `Ts` にそれぞれ設定する。評価区間をいくつかの並列処理に分割するかを `options.DoP` に設定する。シミュレーションにおいて NMPC の最適化計算を行うソルバは `ParNMPC` フォルダにある M ファイル `NMPC_Solve.m` である。これが問題依存の M ファイルを呼び出して最適

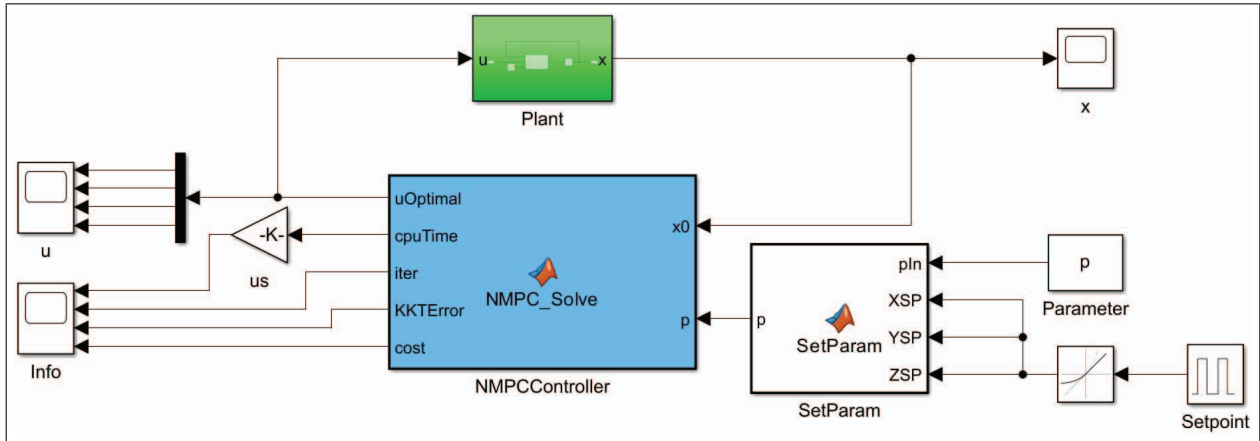


図4 Simu_Simulink.slx

生成されたソルバのライブラリを制御器の **NMPCController** ブロックとして使用し制御対象の **Plant** ブロックと結合した閉ループ系のシミュレーションを行う。参照信号を外部信号として与えている。x, u, Info の Scope ブロックを開くとシミュレーション結果の時間履歴が表示される。

化計算を行う。シミュレーション結果のデータは MAT ファイル GEN_log_rec.mat に保存される。

Simu_Matlab.Codegen.m を実行すると Simu_Matlab.m から C コードが生成、ビルドされ、実行可能ファイル Simu_Matlab.exe が生成される。これを OS 上で実行してもよいし、MATLAB コマンドラインで !Simu_Matlab と入力して実行することもできる。Simu_Matlab.m と同じシミュレーションだが大幅に高速化される。シミュレーション結果はテキストファイル GEN_log_rec.txt に保存される。

2.5 Simulink によるシミュレーションと MEX 関数の生成

閉ループ系のシミュレーションは Simulink 上でも行うことができ、そのためのライブラリを生成するのが Simu_Simulink_Setup.m である。並列処理の分割数 options.DoP を設定して Simu_Simulink_Setup.m を実行すると、ParNMPC フォルダの M ファイル NMPC_Solve.m から C コードが生成、ビルドされ、ライブラリが生成される。そして、Simulink ブロック線図 Simu_Simulink.slx のパラメータが設定され Simu_Simulink.slx が Simulink エディタで開かれる(図4)。Simu_Simulink.slx を実行すると、その中の NMPCController ブロック内で NMPC_Solve ライブラリが呼び出され、制御対象の Plant ブロックと結合した閉ループ系のシミュレーションが実行される。この例では、問題設定の時変パラメータ p に相当する参照値 XSP, YSP, ZSP が外部信号によって与えられている。x, u, Info の Scope ブロックを開くとシミュレーション結果の時間履歴が表示される。

一方、Simu_Simulink_Setup.m でコード生成を行う際に関数

```
NMPC_Solve_CodeGen('dll','C',options);
```

を

```
NMPC_Solve_CodeGen('mex','C',options);
```

と変更して実行すると、NMPC_Solve.m から C コードが生成、ビルドされ、MEX ファイル NMPC_Solve_mex.mexw64 が生成される。これによって MEX 関数 NMPC_Solve_mex が NMPC_Solve と同じ処理を行う関数として呼び出せるようになる。たとえば、Simu_Matlab.m のシミュレーションループで呼び出される

```
[solution,output]
= NMPC_Solve(x0Measured,p,options);
```

を

```
[solution,output]
= NMPC_Solve_mex(x0Measured,p,options);
```

に変更すると、MEX 関数を使用した MATLAB シミュレーションが実行できる。MEX 関数で最適化計算を行うことにより処理が大幅に高速化できる。

並列処理の分割数 options.DoP を変えて閉ループ系 10 秒間のシミュレーションを時間刻み 0.01 秒で行い、

- i) Simu_Matlab.m, NMPC_Solve
- ii) Simu_Matlab.m, NMPC_Solve_mex
- iii) Simu_Matlab.exe

それぞれに対して、平均反復回数ならびに実行時間のシミュレーション時間に対する比を調べたところ表1のようになった。ただし、後者については OS による処理の影響を除くため 10 回実行した場合の最小値を示している。この比が 1 以下であれば、制御入力更新 1 回あたりの平均計算時間がシミュレーションの時間刻みを下回り、時間刻みと同程度の制御周期で実装が可能ということになる。表1より、並列処理のための近似を導入しても平均反復回数はほとんど増えないことがわかる。また、MEX 関数や実行可能ファイルへ変換することで計算が大幅に高速化でき実時間実装が十分に可能であることと、変換

表 1 平均反復回数ならびに実行時間とシミュレーション時間の比

実行時間とシミュレーション時間の比が 1 以下であれば制御入力更新 1 回あたりの平均計算時間がシミュレーションの時間刻みを下回ることになり、時間刻みと同程度の制御周期で実装が可能である。

options.DoP	1	2	4	8	16
平均反復回数	2.49	2.50	2.52	2.80	2.83
Simu_Matlab.m, NMPC.Solve	1.2	1.3	1.4	1.7	1.9
Simu_Matlab.m, NMPC.Solve_mex	0.060	0.048	0.036	0.038	0.039
Simu_Matlab.exe	0.034	0.023	0.021	0.020	0.021

後は利用可能なスレッド数に応じて高速化できていることも確認できる。最適化ソルバのみを MEX 関数に変換した場合、シミュレーション全体を変換した場合ほど高速化されないが、実機の制御系などで容易に再利用できる。ほかの例題での評価やほかの数値解法との比較については文献 9) を参照されたい。

3. おわりに

本稿では、近年注目されている MPC の実装を容易にする自動コード生成ツールの中で、並列計算を活用する ParNMPC を紹介した。試してみようと思っていたのであれば幸いである。ほかにも、Jupyter Notebook 上で Python を利用して C++ コードを生成するツール¹³⁾や、剛体リンク系の効率的な動力学アルゴリズムのライブラリを活用しロボットシステムに特化したツール¹⁴⁾も開発されている。また、折しも来年 NMPC に関する国際会議が京都で開催される¹⁵⁾。研究コミュニティがさらに活性化し、従来の限界を超え常識を変える数値解法とツールが開発されていくことを期待したい。

(2023 年 5 月 30 日受付)

参 考 文 献

1) <https://www.bostondynamics.com/resources/blog/picking-momentum> (2023 年 5 月 29 日閲覧)
2) <https://toyotatimes.jp/en/spotlights/1005.html> (2023 年 5 月 29 日閲覧)

3) <https://nips.cc/virtual/2021/tutorial/21892> (2023 年 5 月 29 日閲覧)
4) Special Issue: Online Motion Planning and Model Predictive Control, *Advanced Robotics*, **37**–5 (2023)
5) 特集：実応用こそ非線形制御理論、計測と制御, **61**–2 (2022)
6) 大塚敏之：非線形モデル予測制御の研究動向，システム／制御／情報, **61**–2, 42/50 (2017)
7) 大塚敏之（編）：実時間最適化による制御の実応用，コロナ社 (2015)
8) R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl: acados — A Modular Open-Source Framework for Fast Embedded Optimal Control, *Mathematical Programming Computation*, **14**–4, 147/183 (2022)
9) H. Deng and T. Ohtsuka: ParNMPC — A Parallel Optimization Toolkit for Real-Time Nonlinear Model Predictive Control, *International Journal of Control*, **95**–2, 390/405 (2022)
10) <https://deng-haoyang.github.io/ParNMPC/> (2023 年 5 月 29 日閲覧)
11) <https://github.com/deng-haoyang/ParNMPC> (2023 年 5 月 29 日閲覧)
12) H. Deng and T. Ohtsuka: A Parallel Newton-Type Method for Nonlinear Model Predictive Control, *Automatica*, **109**, 108560 (2019)
13) S. Katayama and T. Ohtsuka: Automatic Code Generation Tool for Nonlinear Model Predictive Control with Jupyter, *IFAC-PapersOnLine*, **53**–2, 7033/7040 (2020)
14) <https://github.com/mayataka/robotoc> (2023 年 5 月 29 日閲覧)
15) <https://nmppc2024.org/> (2023 年 5 月 29 日閲覧)

[著 者 紹 介]

おお つか とし めき
大 塚 敏 之 君 (正会員)



1995 年東京都立科学技術大学大学院工学研究科工学システム専攻博士課程修了。同年筑波大学構造工学系講師。1999 年大阪大学大学院工学研究科講師，2003 年同助教授，2007 年同大学大学院基礎工学研究科教授，2013 年京都大学大学院情報学研究科教授，現在に至る。主として、非線形制御と最適制御の理論と応用に関する研究に従事。博士（工学）。システム制御情報学会，日本航空宇宙学会，日本機械学会などの会員。AIAA，IEEE のシニア会員。